

CLAUDE CODE ADVANCED: THE BROWSER, THE PHONE, THE COMPANY

Two Models, One Job

Claude Code Advanced, episode 6

Two models agreeing is not two checks. It is two guesses that happen to match.

You asked Claude a hard question, pasted the answer into ChatGPT, and asked whether it was right. ChatGPT said yes, you felt better, and you shipped it. Two models agreed. Does that make it true?

This sheet is how to run two models so that the second one earns its cost, and how to avoid the trap that makes most multi-model workflows a confidence machine rather than a checking one.

Claude Code changes with every release. Every figure, default and feature name on this sheet was checked on the date below and is stated as of that date. Check the current documentation before you rely on any of it.

In scope

- Why a second model helps, and which reason actually matters
- Director and executor, and why the split has to be asymmetric
- Adversarial review, and the prompt that does it
- Why only artefacts travel between two models
- Agreement, disagreement, and which one is information
- What not to paste into a second vendor
- When two models is worth it and when it is theatre

Out of scope

- Which model is better, which is the next episode
- Benchmarks, which age badly and rarely predict your work
- Any claim that two models are more accurate than one. They are not
- Automated multi-model voting, which compounds the trap rather than fixing it
- Tooling comparisons and setup guides, which date faster than the argument

Nearly everything here is operational judgement rather than vendor documentation, and it is labelled that way throughout. A page about not laundering opinion as fact has to hold itself to that first.

01 When a second model earns its cost

Attacking a plan before you build it	usually worth it
A design you will live with for a year	worth it
A decision that is expensive to reverse	worth it
A claim you are about to say out loud to an audience	worth it
Routine work with good context	one model, every time
The test	would you change your mind based on the answer?

A judgement, not a measurement. The test in the last row is the only one that reliably separates the first four from habit.

03 Why two at all, and which reason matters

The obvious reason is independent failure modes: a model trained differently does not share the first one's blind spot, so the things it misses are a different set of things.

The reason that actually matters is sunk cost. The second model has NONE. It did not spend forty minutes arguing itself into the current design, it has no attachment to the approach, and no memory of why the awkward compromise seemed reasonable at the time.

That is a property you cannot buy from the first model at any price, because you built the sunk cost together. It is also why "start a fresh session" works within one model: same mechanism, smaller scale.

A second model has no sunk cost in the work already done, which is the property that makes it useful as a reviewer.

`Operational practice, not vendor documentation`

Starting a fresh session gives clean context aimed entirely at the task, which is the same mechanism applied within one model.

`Claude Code docs, Best practices, checked September 2026`

04 Director and executor, with a worked example

The division that works is that one model DIRECTS and reviews while the other EXECUTES with tools. The executor has the codebase, the terminal and the ability to run things. The director has distance.

A concrete shape for it: you are extracting a billing service out of a monolith. The director never sees the repository. It gets the current call graph, the invariants that must survive, and the deadline, and it produces a sequence of migrations with the risk on each one named. The executor gets that sequence, opens the code, runs the tests, and reports what it found that the plan did not anticipate.

The reason the split works is symmetrical: the executor has everything except distance, and the director has nothing except distance.

The director works from a description and the executor works from the repository. The asymmetry is the point: a reviewer who has been editing the file all afternoon cannot see it any more.

`Operational practice, not vendor documentation`

05 Attack the plan, do not review it

Give the second model the first one's PLAN, not its conclusion, and ask it to attack. Not review. Attack. Where does this fail, what did it assume, what did it not consider.

A reviewer with nothing invested finds a different class of problem from the one that wrote it, and finds it faster, because it is not defending anything.

The documented caveat applies exactly as hard across vendors as within one: a reviewer prompted to find gaps will usually report some, even when the work is sound, because that is what it was asked to do. Chasing every finding leads to over-engineering. Ask it to flag only what affects correctness or the stated requirements.

A reviewer prompted to find gaps will usually report some, even when the work is sound, because that is what it was asked to do. Chasing every finding leads to over-engineering.

Claude Code docs, Best practices, checked September 2026

06 Only artefacts travel

A conversation cannot leave the tool it happened in. What moves between two models is artefacts: a prompt, a spec written to a file, a diff, a failing test, an error message.

That is the entire transfer protocol between any two models from any two vendors, and it is not going to change.

Which is why this series treats prompts as code. A prompt you can paste into a different tool is portable. A clever forty-message conversation is not. If you want two models working on one job, the job has to exist in FILES.

The most useful specs are self-contained: they name the files and interfaces involved, state what is out of scope, and end with an end-to-end verification step.

Claude Code docs, Best practices, checked September 2026

Only artefacts move between models. This is an observation about how the tools work, not a documented feature.

First principles, not vendor documentation

07 Agreement is not verification

This is the centre of the page. Two models agreeing is not two checks. It is two guesses that happen to match, produced by systems trained on overlapping data, which fail in correlated ways more often than anyone would like.

Pasting the second model's approval into your pull request as evidence is not checking. It is LAUNDERING: you took an unverified claim, ran it past something that also cannot verify it, and came back with a stronger feeling and exactly the same amount of evidence.

The confidence went up. The truth did not move. And the more automated the arrangement, the worse this gets, which is why multi-model voting schemes are not on this page.

Agreement between two models is not independent verification, because their failure modes are correlated rather than independent.

First principles, not vendor documentation

08 Disagreement is the information

When the two disagree, that is genuinely useful, and useful in a specific way: it names the exact spot where you now have to consult a primary source.

Not resolve it by asking a third model. Not go with whichever sounded more confident. Read the documentation, run the thing, or write the test that settles it.

Disagreement points at where the uncertainty actually lives. Agreement tells you almost nothing, which is uncomfortable, because agreement is the outcome that feels like success.

What to do instead: write every factual claim down with the page it came from and the date it was checked, and label anything that cannot be traced to a primary source as a judgement, out loud, in the same file. Treat both models' output as a draft.

Disagreement between models identifies where to consult a primary source. Agreement does not constitute evidence.

Operational practice, not vendor documentation



09 A second vendor is a second set of terms

The moment you involve a second provider you have two sets of terms, two privacy policies, two retention rules and two organisations holding whatever you sent.

So: no secrets, no credentials, no keys. No customer data. No proprietary code your employer would not want sitting in a second provider's logs.

This is not paranoia and it is not a criticism of either vendor. It is the ordinary observation that "paste it into the other one and see what it says" is a data transfer, and deserves the same thought as any other data transfer.

Sending content to a second provider is a data transfer subject to that provider's terms. This is a contractual observation, not a product claim.

First principles, not vendor documentation

10 What it costs, honestly

You pay twice in money, which is obvious, and twice in the expensive currency, which is your own attention: two conversations to hold, two sets of context to maintain, and a reconciliation step that did not exist before.

Sometimes that is clearly worth it. A design you will live with for a year. A decision expensive to reverse. A claim you are about to say out loud to an audience.

And sometimes it is theatre: running everything past a second model so you can feel checked, while adding nothing but latency and a bill. The test that separates them is whether you would actually change your mind based on the answer.

The honest limit, which sounds like a downgrade and is not: for most tasks, ONE model with good context beats two with poor context. Context is still the lever.

One model with good context outperforms two with poor context on most tasks, because context is the dominant variable.

Operational practice, not vendor documentation

What you get back is mostly decided by what is in the context window when you ask.

Claude Code docs, How Claude Code works, checked September 2026

Before you reach for the second model

1. Would I change my mind based on the answer? If not, do not ask
2. Am I giving it the PLAN, or just the conclusion?
3. Did I ask it to attack rather than review?
4. Did I tell it to flag only what affects correctness?
5. Am I treating disagreement as the signal, not as a tie to break?
6. When they disagreed, did I go and read a primary source?
7. Am I about to paste a secret, customer data, or proprietary code?
8. Is the work in files, so it can actually move between them?
9. Would one model with better context have done this?
10. Am I recording the source and the date, or just the verdict?

Everything else

Site

<https://code-with-sam-dev.github.io>

YouTube

<https://www.youtube.com/@CodewithSam-Dev>

LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>

X

<https://x.com/CodeWithSamDev>

TikTok

<https://www.tiktok.com/@codewithsamdev>

Instagram

<https://www.instagram.com/codewithsamdev>

GitHub

<https://github.com/code-with-sam-dev>

Best practices

<https://code.claude.com/docs/en/best-practices>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Claude and Claude Code are products of Anthropic. This is an independent, unofficial guide produced by Code with Sam. It is not affiliated with, endorsed by, or sponsored by Anthropic, and no Anthropic artwork is reproduced here. Product names appear only to identify the software being discussed. Other names are the property of their respective owners.

Technical claims verified 2026-09-13 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.